

Machine Learning Final Assignment

Stephen Haskins: 21365534

January 5, 2026

Introduction

This assignment explores the repurposing of a GPT transformer model to solve mathematical and boolean expressions. Each of the two sections involve building appropriate datasets, defining evaluation metrics, adapting model architectures, and analysing results across various arithmetic operations.

Part 1: Math GPT

1.1: Build an appropriate dataset

Designing an appropriate dataset is crucial for the performance of the Math GPT model. The dataset was generated via a python script that deterministically creates a series of unique arithmetic problems and their respective solutions. This deterministic approach ensures all ground truth labels are completely truthful. The dataset includes a variety of operations such as addition, subtraction, multiplication, and division, with two levels of complexity selected via the ‘-basic’ and ‘-advanced’ flags.

Basic Expressions

The -basic flag generates expressions involving single-digit operands and simple operator combinations. This mode was used during early experimentation to validate that the transformer model, tokenisation method, and evaluation scripts were all functioning as expected.

Advanced Expressions

The -advanced flag extends this setup to more challenging two-digit arithmetic, including larger operand ranges and higher-magnitude outputs. This mode is intended to test the model’s ability to generalise to more complex mathematical tasks, and is used for final evaluation. Both levels of complexity are shown below in Table 1

Basic examples	Advanced examples
$16 \times 31 = 496$	$52 \times 40 + 2 = 2082$
$77 - 50 = 27$	$5 - 9 \times 3 = -22$
$98 + 44 = 142$	$(1 \times 1) + 1 = 2$
$43 + 98 = 141$	$4 + (9 \times 2) = 22$
$93 - 48 = 45$	$65 + (4 \times 26) = 169$

Table 1: Example outputs for basic and advanced mathematical datasets

The dataset is then split into training and test sets using holdout validation with a 80/20 split. This ensures that evaluation is performed on unseen expressions, and provides a fair and reliable measure of model performance.

Shortcomings of the Dataset

One pitfall of this dataset design is that the division expression is under-represented. This is due to generating integer-only results, which significantly reduces the number of valid division

expressions that can be generated. This constraint was added to avoid decimal results, which would add significant and disproportionate complexity to the division operation.

1.2: Define appropriate evaluation metrics

To evaluate the performance of the Math GPT model, multiple metrics were recorded to capture different aspects of the model’s performance. The primary metric used was accuracy, defined as the percentage of correctly predicted expressions in the test set. This is the most task-appropriate metric, as the goal is to have the model produce exact matches to the ground truth solutions.

To provide further granularity to this metric, operation-wise accuracy was also recorded. This breaks down the overall accuracy by arithmetic operation, revealing whether performance is uniform across operations, or whether the model finds certain operations more challenging.

The Character Error Rate (CER) was implemented retrospectively as results showed predictions were often off by just one or two digits. This metric captures how many single-character edits are required to transform the model output into the correct answer, meaning: CER represents near-miss errors, such as predicting 1025 instead of 1024, as opposed to complete hallucinations.

Finally, the Mean Absolute Error (MAE) measures the average absolute difference between the predicted and true numeric values. This provides insight to how far incorrect predictions deviate from the ground truth.

Using the above string and numerical based metrics in conjunction, provides a balanced and reliable evaluation framework for this assignment.

1.3: Explore adaptations of the model architecture for this application

Model Capacity Reduction

The initial model configuration had a character-level transformer design, with a block size of 256, an embedding dimension of 384, and 6 transformer layers, resulting in 10.74 million parameters. Arithmetic expressions are short, structured sequences with limited contextual dependency, and therefore do not require long receptive fields or deep hierarchical representations.

Thus, to better align model with task requirements, the architecture was substantially down-scaled. The block size was reduced from 256 to 32, the embedding dimension from 384 to 256, and the number of layers from 6 to 4, resulting in a model with approximately 3.15 million parameters. This reduction significantly decreased computational cost, enabling longer training runs and more stable optimisation.

The impact of this architectural change is evident in the quantitative results in Table 2. The original model was only trained for 1,000 iterations due to time and resource constraints, and exhibited very poor performance, achieving an accuracy of only 3.11%. Performance was poor across all operators, with particularly weak results on multiplication. The high character error rate (0.7802) further suggests that most predicted sequences were structurally incorrect.

In contrast, the downscaled model was trained for 5,000 iterations and achieved an accuracy of 59.00%. Substantial improvements were observed across all operators except multiplication, which remained comparatively difficult. The character error rate decreased to 0.2338, indicating that the majority of output sequences were largely correct at the character level. The mean absolute error also decreased by more than 60%, demonstrating improved numerical accuracy in cases where predictions were not exact.

These results indicate that reducing model capacity was critical for effective learning in this setting. The smaller model not only reduced over-parameterisation, but also allowed sufficient optimisation time to learn arithmetic structure, resulting in large gains in both symbolic and

numeric accuracy.

Metric	Original Model	Downscaled Model
Training Iterations	1,000	5,000
Parameters (millions)	10.74	3.15
Block Size	256	32
Embedding Dimension	384	256
Transformer Layers	6	4
Overall Accuracy	3.11%	59.00%
Addition Accuracy	5.45%	80.47%
Subtraction Accuracy	2.88%	75.78%
Multiplication Accuracy	0.30%	10.00%
Division Accuracy	16.98%	71.93%
Character Error Rate	0.7802	0.2338
Mean Absolute Error	270.45	107.94

Table 2: Numerical comparison between the original and downscaled model architectures

Early Stopping

Although the downscaled model converged effectively, prolonged training revealed clear signs of overfitting. Early in training, both training and validation loss decreased steadily, indicating successful learning of the underlying arithmetic patterns. However, after convergence, training loss continued to decrease marginally while validation loss increased substantially. This divergence indicates that the model began to memorise training examples rather than improving generalisable arithmetic reasoning. This behaviour can be seen in Table 3 below.

To mitigate this behaviour, early stopping was introduced. Training was halted once validation loss plateaued, preventing further degradation in generalisation performance. This reduced unnecessary computation and reduced overfitting.

Training Stage	Training Loss	Validation Loss
Step 0	2.8065	2.8149
Step 25	1.9796	2.0290
Step 1910	0.2377	3.5586
Step 1920	0.2343	3.6422
Step 1930	0.2328	3.6369

Table 3: Training and validation loss at early and late stages of optimisation.

1.4: Explore results for a range of arithmetic options

Results are shown under the downscaled model in Table 2 above.

Addition

Addition is the best-learned operation, achieving an accuracy of 80.47%. This strong performance could be due to the relatively local and regular structure of addition. Multi-digit addition can commonly be decomposed into digit-wise operations with short dependency chains, where carries typically affect only adjacent digits. For example, expressions such as $12+13=25$ are frequently solved correctly. However, as the complexity of the addition increases, such as in $98+76=174$, the model struggles more, likely due to the longer dependency chains and carry operations involved.

Subtraction

Subtraction achieves the second-highest accuracy at 75.78%. This slightly weaker performance is expected, given the added complexity of borrowing compared to carrying, as well as sign handling for negative results. The model performs well on simpler cases such as $54-20=34$, but errors are more common when multiple borrow operations or negative results are involved, for example in expressions like $23-89$.

Division

Division achieves an accuracy of 71.93%, which is lower than addition and subtraction but still demonstrates non-trivial learning. As mentioned, division is under-represented in the dataset, with only 53 division examples appearing in the test set, compared to roughly one thousand examples for each of the other operators. Typical correctly solved cases include expressions such as $48/6=8$ or $81/9=9$. Given the small sample size, this accuracy estimate is noisier and less reliable, and performance would be expected to degrade significantly if non-integer quotients or decimal representations were introduced.

Multiplication

Multiplication is the weakest operation by a large margin, with an accuracy of just 10.00%. This behaviour is expected and highlights a limitation of the model. Multi-digit multiplication requires quadratic interactions between digits, ie: every digit in operand A interacts with every digit in operand B. As well as this, long dependency chains due to multiple carries, and the generation of comparatively long outputs complicates the multiplication operation significantly. For example, in the case $53*71=3763$, the model must correctly generate all four digits in the sequence, and a single incorrect carry can corrupt the entire result. Results show, the model often produces outputs that are numerically incorrect but with near-miss predictions that differ by one or two digits. For this reason, the CER metric was used.

CER and MAE

The character error rate of 0.2290 indicates that, on average, roughly 23% of characters in the model's predicted answers differ from the ground-truth output. This reflects errors at the character level rather than purely numeric correctness. Multiplication is expected to dominate this metric, as its longer outputs make it especially sensitive to cascading mistakes. Creating a more detailed analysis on CER, such as per-operation, and reducing character-level errors to improve overall accuracy would be the first point of future work.

The mean absolute error of 116.94 highlights a similar asymmetry across operations. Errors in addition and subtraction typically produce small numeric deviations, whereas multiplication errors often lead to large numerical discrepancies. A single incorrect digit in a higher place value can result in errors of hundreds or more, causing multiplication to disproportionately inflate MAE despite being only one category of operation.

Part 2: Boolean GPT

2.1: Build an appropriate dataset

Similar to the math model, the Boolean GPT model's dataset was generated deterministically via a python script. This script creates unique boolean expressions and their respective solutions, ensuring ground truth labels are completely accurate. The dataset includes boolean logical operations such as AND, OR, NOT, and XOR, with two levels of complexity selected via the

‘-basic’ and ‘-advanced’ flags. Similar to before, the basic dataset was just used to confirm the model and evaluation scripts were functional, while the advanced dataset was used for final evaluation.

Issues with dataset

Due to the nature of Boolean logic, the number of unique expressions is much far than in the previous math dataset. This is due to the nature of Boolean logic: There are only two operands, True and False, and a limited set of operators (AND, OR, XOR, NOT), meaning that all possible expressions can be exhaustively enumerated with relatively few samples. As a result, the dataset contains only a few hundred unique expressions. Increasing the dataset size would only be possible by scaling the complexity of expressions to a much greater degree.

Finally, because the model is character-level, accuracy is sequence-based rather than label-based. The model must generate the exact target string character-by-character, so even a logically correct intention that produces an incorrect sequence (e.g., wrong spelling, extra whitespace, or partial tokens) is treated as incorrect. This issue is accounted for in the upcoming architectural adaptations section.

2.2: Define appropriate evaluation metrics

Unlike math arithmetic, boolean logic has either a True or False output, meaning this task can be treated as a binary classification problem.

The primary metric is accuracy, defined as the proportion of test expressions for which the model’s generated output matches the ground-truth Boolean value. This is the most appropriate headline metric for Boolean logic, as any deviation from the correct value represents a complete logical failure rather than a partial error. Because the output space is binary and unambiguous, accuracy directly measures task success.

To gain more detailed insight into model behaviour, operator-wise accuracy is also reported. This metric breaks accuracy down by the logical operator appearing in each expression (AND, OR, XOR, NOT). This analysis is particularly important for Boolean logic, as different operators have fundamentally different properties. Operator-wise accuracy helps identify whether the model has learned general logical reasoning or is relying on basic shortcuts that work only for certain operators.

In addition, standard classification metrics derived from the confusion matrix were computed: precision, recall, and F1 score. These metrics complement accuracy by exposing potential biases in the model’s predictions. For example, a model that defaults to predicting False on difficult expressions could achieve deceptively high accuracy if the dataset is balanced, but would exhibit poor recall for the True class. Precision, recall, and F1 score make such failure modes explicit and ensure that performance is not being overstated by a single metric.

Finally, to provide even further granularity to the accuracy metric, operation-count accuracy was also recorded, differentiating expressions with one operator from those with 2.

Together, these metrics provide a rigorous and interpretable evaluation framework for assessing the Boolean GPT model’s performance.

2.3: Explore adaptations of the model architecture for this application

Model Downsizing

Boolean expressions are short, discrete, and deterministic, with a small fixed vocabulary and shallow compositional structure. As such, the initial transformer configuration was far too complex for this domain.

The original GPT model used a block size of 256, an embedding dimension of 384, 6 attention heads, and 6 transformer layers. While expressive, this configuration introduced unnecessary computational overhead and increased the risk of overfitting. To better align model capacity with task complexity, the architecture was substantially downscaled. The final configuration reduced the block size to 32, the embedding dimension to 128, the number of attention heads to 4, and the number of layers to 4.

This downsizing enabled longer training (5,000 iterations instead of 1,000) within practical time constraints and improved stability. Empirically, this change led to higher accuracy, confirming that Boolean logic evaluation benefits from compact, shallow architectures rather than deep transformers.

Input Representation

Despite these improvements, early results revealed partial prediction failures when using the full textual representation of Boolean values and operators. In particular, the model frequently produced incomplete or truncated outputs, such as predicting “lse” instead of “False” or “ue” instead of “True”. These errors indicate that the model struggled with longer token sequences and character-level alignment, rather than with the underlying logical structure.

To address this issue, the input representation was simplified. Boolean constants were mapped from textual tokens (“True”, “False”) to single-character symbols (“1”, “0”), and logical operators (e.g., “AND”, “OR”, “XOR”, “NOT”) were replaced with their symbolic equivalents (“&”, “|”, “^”, “!”). This transformation significantly reduced sequence length, vocabulary size, and output ambiguity, while preserving the exact logical semantics of each expression.

The effect of this representational change was substantial, as shown in Table 4. Using the full textual representation, the model achieved an overall accuracy of approximately 38%, indicating that more than half of the predictions were incorrect. Character error rates remained high, and performance degraded rapidly as expression complexity increased.

In contrast, the simplified symbolic representation yielded an overall accuracy exceeding 75%. Performance improved consistently across operators and expression complexities, confirming that the model’s earlier failures were primarily due to representational inefficiencies rather than an inability to learn Boolean logic itself.

These results show that, for Boolean logic evaluation, representation choice is as critical as architectural design. While downsizing the transformer improved efficiency and convergence, simplifying the input representation addressed the dominant source of error by reducing sequence length and eliminating partial-token failure modes. The resulting model is both more accurate and more robust, supporting the conclusion that symbolic reasoning tasks benefit from compact architectures paired with minimal, semantically aligned vocabularies.

Metric	Full Text Representation	Symbolic Representation
Overall Accuracy	38.00%	75.51%
Precision	0.9091	0.7391
Recall	0.4167	0.7391
F1 Score	0.5714	0.7391

Table 4: Comparison of performance using full textual and simplified representations.

2.4: Explore results for a range of arithmetic options

Evaluation on the unseen test expressions yields an overall exact match accuracy of 75.51%.

Operator	Full Text Accuracy	Symbolic Accuracy
AND (&)	40.00%	69.23%
OR ()	57.14%	100.00%
XOR (^)	32.14%	72.41%
NOT (!)	40.00%	100.00%

Table 5: Operator-wise accuracy for Boolean expressions under different input representations.

OR operator

Operator-wise results show clear variation in difficulty. OR is learned perfectly, achieving 100% accuracy, with expressions such as $1 | 0=1$ and $(0 | 0) | 0=0$ always evaluated correctly. This reflects the simplicity and monotonic structure of OR, which is easy for the model to capture even in chained expressions.

AND operator

AND also performs well, with an accuracy of 76.92%, and is generally correct in cases like $1 \& 0 \& 0=0$. Errors typically arise when AND is combined with other operators, indicating challenges with composition rather than misunderstanding AND itself.

NOT operator

NOT is also solved perfectly in this evaluation, but it is under-represented, appearing in only a handful of test cases. While simple negation such as $! 0=1$ is handled correctly, this result should be interpreted cautiously due to the small sample size.

XOR operator

XOR is the weakest operator, with an accuracy of 68.97%. Many errors involve chained or nested XOR expressions, such as $1 \wedge 1 \wedge 1$ or $0 \wedge (1 \wedge 0)$, reflecting the non-linearly separable nature of XOR and the need to track parity across operands. This makes XOR substantially harder than AND or OR for the model.

Expression complexity further explains these results. All single-operator expressions are solved correctly, while accuracy drops to 73.33% for expressions containing two operators. This indicates that the main failure mode is compositional reasoning, particularly when multiple operators interact.

Precision, recall, and F1 score are all 0.7391, indicating balanced performance across True and False classes. Overall, the model reliably learns simple Boolean operators, but struggles with XOR and with deeper compositional structure, highlighting the limits of character-level reasoning for non-linear logical operations.

Part 3: Discussion

This assignment compared two symbolic reasoning tasks implemented using the same character-level transformer framework. While both tasks are deterministic and rule-based, their structural properties differ, leading to distinct architectural and representational requirements.

A key similarity across both models is that large, language-style transformers were unnecessary. In both cases, substantial model downsizing proved optimal: fewer layers, smaller embedding sizes, and shorter context windows consistently improved performance. This reflects the limited depth and short-range dependencies inherent to symbolic expressions. Over-parameterised

models not only increased computational cost but also caused overfitting. Early stopping was therefore essential in the Math GPT setting to preserve generalisation once validation loss began to diverge from training loss.

The two tasks differed in where their difficulties arose. In Math GPT, the dominant challenge was algorithmic complexity. Operations such as addition and subtraction were learned reliably, while multiplication remained extremely difficult due to long dependency chains, quadratic digit interactions, and cascading carry errors. These issues were reflected in low exact-match accuracy but more moderate character error rates (CER), motivating the use of CER and MAE as complementary evaluation metrics. These metrics revealed that many failures were near-miss numerical errors rather than arbitrary outputs, providing a more nuanced assessment of model behaviour.

In contrast, Boolean GPT was limited less by reasoning complexity and more by representation. When trained on full textual tokens (“True”, “False”, “AND”, etc.), the model frequently failed due to partial-token generation errors, despite often capturing the correct logical structure. Simplifying the representation to a compact symbolic vocabulary dramatically improved accuracy, demonstrating that for Boolean logic, sequence length and token alignment dominate performance. This highlights a fundamental difference between the tasks: Boolean reasoning is logically simpler but more sensitive to surface form at the character level.

Evaluation strategies also reflected these differences. Accuracy was appropriate for both tasks, but Math GPT benefited from graded error metrics (CER, MAE), whereas Boolean GPT could be treated as a binary classification problem with precision, recall, and F1 scores. Operator-wise breakdowns in both cases were informative, consistently identifying multiplication and XOR as the hardest operators due to their non-linear or non-local structure.

Overall, these experiments show that transformer-based models can learn symbolic reasoning patterns, but only when architectural capacity and representation are matched to task structure. Remaining limitations, particularly multiplication and deep compositional Boolean expressions, suggest that character-level transformers struggle with algorithmic reasoning. Future work could explore digit-level tokenisation or curriculum learning to better capture these underlying structures.

Appendix: Code

```
#!/usr/bin/env python3

import torch
import torch.nn as nn
from torch.nn import functional as F
import argparse
import json
import re
from pathlib import Path

# hyperparameters
# Defaults set to "Target" values
batch_size_default = 64
block_size_default = 256
max_iters_default = 1000
eval_interval_default = 500
learning_rate_default = 3e-4
```

```

n_embd_default = 384
n_head_default = 6
n_layer_default = 6
dropout_default = 0.2
device = 'mps' if torch.backends.mps.is_available() else 'cpu'
eval_iters = 500

# -----

print(f"Using device: {device}")

torch.manual_seed(1337)

# these were used for a grid search
parser = argparse.ArgumentParser()
parser.add_argument('--dataset', type=str, default='input_math.txt')
parser.add_argument('--model_name', type=str, nargs='?', default='gpt_math_model.pth')
parser.add_argument('--max_iters', type=int, default=max_iters_default)
parser.add_argument('--batch_size', type=int, default=batch_size_default)
parser.add_argument('--block_size', type=int, default=block_size_default)
parser.add_argument('--eval_interval', type=int, default=eval_interval_default)
parser.add_argument('--learning_rate', type=float, default=learning_rate_default)
parser.add_argument('--n_embd', type=int, default=n_embd_default)
parser.add_argument('--n_head', type=int, default=n_head_default)
parser.add_argument('--n_layer', type=int, default=n_layer_default)
parser.add_argument('--dropout', type=float, default=dropout_default)
args = parser.parse_args()

batch_size = args.batch_size
block_size = args.block_size
max_iters = args.max_iters
eval_interval = args.eval_interval
learning_rate = args.learning_rate
n_embd = args.n_embd
n_head = args.n_head
n_layer = args.n_layer
dropout = args.dropout

print(f"Loading dataset from {args.dataset}")
print(f"Model will be saved as {args.model_name}")
print(f"Training for {args.max_iters} iterations")

BASE_DIR = Path(__file__).resolve().parent
DATASET_DIR = BASE_DIR / "datasets"
dataset_path = DATASET_DIR / args.dataset

with open(dataset_path, 'r', encoding='utf-8') as f:
    text = f.read()

```

```

# here are all the unique characters that occur in this text
chars = sorted(list(set(text)))
vocab_size = len(chars)
# create a mapping from characters to integers
stoi = { ch:i for i,ch in enumerate(chars) }
itos = { i:ch for i,ch in enumerate(chars) }
encode = lambda s: [stoi[c] for c in s] # encoder: take a string, output a list of integers
decode = lambda l: ''.join([itos[i] for i in l]) # decoder: take a list of integers, output a

# Train and test splits
data = torch.tensor(encode(text), dtype=torch.long)
n = int(0.9*len(data)) # first 90% will be train, rest val
train_data = data[:n]
val_data = data[n:]

# data loading
def get_batch(split):
    # generate a small batch of data of inputs x and targets y
    data = train_data if split == 'train' else val_data
    ix = torch.randint(len(data) - block_size, (batch_size,))
    x = torch.stack([data[i:i+block_size] for i in ix])
    y = torch.stack([data[i+1:i+block_size+1] for i in ix])
    x, y = x.to(device), y.to(device)
    return x, y

@torch.no_grad()
def estimate_loss():
    out = {}
    model.eval()
    for split in ['train', 'val']:
        losses = torch.zeros(eval_iters)
        for k in range(eval_iters):
            X, Y = get_batch(split)
            logits, loss = model(X, Y)
            losses[k] = loss.item()
        out[split] = losses.mean()
    model.train()
    return out

class Head(nn.Module):
    """ one head of self-attention """

    def __init__(self, head_size):
        super().__init__()
        self.key = nn.Linear(n_embd, head_size, bias=False)
        self.query = nn.Linear(n_embd, head_size, bias=False)
        self.value = nn.Linear(n_embd, head_size, bias=False)
        self.register_buffer('tril', torch.tril(torch.ones(block_size, block_size)))

        self.dropout = nn.Dropout(dropout)

```

```

def forward(self, x):
    # input of size (batch, time-step, channels)
    # output of size (batch, time-step, head size)
    B,T,C = x.shape
    k = self.key(x) # (B,T,hs)
    q = self.query(x) # (B,T,hs)
    # compute attention scores ("affinities")
    wei = q @ k.transpose(-2,-1) * k.shape[-1]**-0.5 # (B, T, hs) @ (B, hs, T) -> (B, T, T)
    wei = wei.masked_fill(self.tril[:T, :T] == 0, float('-inf')) # (B, T, T)
    wei = F.softmax(wei, dim=-1) # (B, T, T)
    wei = self.dropout(wei)
    # perform the weighted aggregation of the values
    v = self.value(x) # (B,T,hs)
    out = wei @ v # (B, T, T) @ (B, T, hs) -> (B, T, hs)
    return out

class MultiHeadAttention(nn.Module):
    """ multiple heads of self-attention in parallel """

    def __init__(self, num_heads, head_size):
        super().__init__()
        self.heads = nn.ModuleList([Head(head_size) for _ in range(num_heads)])
        self.proj = nn.Linear(head_size * num_heads, n_embd)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.dropout(self.proj(out))
        return out

class FeedFoward(nn.Module):
    """ a simple linear layer followed by a non-linearity """

    def __init__(self, n_embd):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_embd, 4 * n_embd, bias=False), # Disabled Bias
            nn.ReLU(),
            nn.Linear(4 * n_embd, n_embd, bias=False), # Disabled Bias
            nn.Dropout(dropout),
        )

    def forward(self, x):
        return self.net(x)

class Block(nn.Module):
    """ Transformer block: communication followed by computation """

    def __init__(self, n_embd, n_head):
        # n_embd: embedding dimension, n_head: the number of heads we'd like
        super().__init__()

```

```

        head_size = n_embd // n_head
        self.sa = MultiHeadAttention(n_head, head_size)
        self.ffwd = FeedFoward(n_embd)
        self.ln1 = nn.LayerNorm(n_embd)
        self.ln2 = nn.LayerNorm(n_embd)

    def forward(self, x):
        x = x + self.sa(self.ln1(x))
        x = x + self.ffwd(self.ln2(x))
        return x

class GPTLanguageModel(nn.Module):

    def __init__(self):
        super().__init__()
        # each token directly reads off the logits for the next token from a lookup table
        self.token_embedding_table = nn.Embedding(vocab_size, n_embd)
        self.position_embedding_table = nn.Embedding(block_size, n_embd)
        self.blocks = nn.Sequential(*[Block(n_embd, n_head=n_head) for _ in range(n_layer)])
        self.ln_f = nn.LayerNorm(n_embd) # final layer norm
        self.lm_head = nn.Linear(n_embd, vocab_size)

        # better init, not covered in the original GPT video, but important, will cover in fo
        self.apply(self._init_weights)

    def _init_weights(self, module):
        if isinstance(module, nn.Linear):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)
            if module.bias is not None:
                torch.nn.init.zeros_(module.bias)
        elif isinstance(module, nn.Embedding):
            torch.nn.init.normal_(module.weight, mean=0.0, std=0.02)

    def forward(self, idx, targets=None):
        B, T = idx.shape

        # idx and targets are both (B,T) tensor of integers
        tok_emb = self.token_embedding_table(idx) # (B,T,C)
        pos_emb = self.position_embedding_table(torch.arange(T, device=device)) # (T,C)
        x = tok_emb + pos_emb # (B,T,C)
        x = self.blocks(x) # (B,T,C)
        x = self.ln_f(x) # (B,T,C)
        logits = self.lm_head(x) # (B,T,vocab_size)

        if targets is None:
            loss = None
        else:
            B, T, C = logits.shape
            logits = logits.view(B*T, C)
            targets = targets.view(B*T)
            loss = F.cross_entropy(logits, targets)

```

```

        return logits, loss

def generate(self, idx, max_new_tokens):
    # idx is (B, T) array of indices in the current context
    for _ in range(max_new_tokens):
        # crop idx to the last block_size tokens
        idx_cond = idx[:, -block_size:]
        # get the predictions
        logits, loss = self(idx_cond)
        # focus only on the last time step
        logits = logits[:, -1, :] # becomes (B, C)
        # apply softmax to get probabilities
        probs = F.softmax(logits, dim=-1) # (B, C)
        # sample from the distribution
        idx_next = torch.multinomial(probs, num_samples=1) # (B, 1)
        # append sampled index to the running sequence
        idx = torch.cat((idx, idx_next), dim=1) # (B, T+1)
    return idx

model = GPTLanguageModel()
m = model.to(device)
# print the number of parameters in the model
print(sum(p.numel() for p in m.parameters())/1e6, 'M parameters')

# create a PyTorch optimizer
optimizer = torch.optim.AdamW(model.parameters(), lr=learning_rate)

for iter in range(max_iters):

    # every once in a while evaluate the loss on train and val sets
    if iter % eval_interval == 0 or iter == max_iters - 1:
        losses = estimate_loss()
        print(f"step {iter}: train loss {losses['train']:.4f}, val loss {losses['val']:.4f}")

    # sample a batch of data
    xb, yb = get_batch('train')

    # evaluate the loss
    logits, loss = model(xb, yb)
    optimizer.zero_grad(set_to_none=True)
    loss.backward()
    optimizer.step()

# Save the model
MODELS_DIR = BASE_DIR / "models"
MODELS_DIR.mkdir(parents=True, exist_ok=True)
model_name = args.model_name
if not model_name.endswith('.pth'):
    model_name += '.pth'
model_path = MODELS_DIR / model_name

```

```

print(f"Saving model {args.model_name} to {model_path}")
torch.save({
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'stoi': stoi,
    'itos': itos,
    'chars': chars,
    'hyperparameters': {
        'batch_size': batch_size,
        'block_size': block_size,
        'max_iters': max_iters,
        'learning_rate': learning_rate,
        'n_embd': n_embd,
        'n_head': n_head,
        'n_layer': n_layer,
        'dropout': dropout
    }
}, model_path)

# Save hyperparameters to JSON
hyperparams_path = MODELS_DIR / f"{model_name}_hyperparams.json"
print(f"Saving hyperparameters to {hyperparams_path}")
with open(hyperparams_path, 'w') as f:
    json.dump({
        'batch_size': batch_size,
        'block_size': block_size,
        'max_iters': max_iters,
        'learning_rate': learning_rate,
        'n_embd': n_embd,
        'n_head': n_head,
        'n_layer': n_layer,
        'dropout': dropout,
        'vocab_size': vocab_size,
        'device': device
    }, f, indent=4)

# generate from the model
context = torch.zeros((1, 1), dtype=torch.long, device=device)
print(decode(m.generate(context, max_new_tokens=500)[0].tolist()))

```

Math Dataset Generation

```
#!/usr/bin/env python3
```

```

import argparse
from pathlib import Path
import random

```

```

# argparse practice
parser = argparse.ArgumentParser(description="Generate a simple math dataset.")
parser.add_argument("--basic", action="store_true", help="Generate basic single-digit math pr

```

```

parser.add_argument("--advanced", action="store_true", help="Generate advanced two-digit math")
parser.add_argument("--split", type=float, default=0.9, help="Train/Test split ratio (default 0.9)")
args = parser.parse_args()

```

```

BASE_DIR = Path(__file__).resolve().parent.parent
DATASET_DIR = BASE_DIR / "datasets"
TRAIN_FILE = DATASET_DIR / "input_math_train.txt"
TEST_FILE = DATASET_DIR / "input_math_test.txt"

```

```

def generate_addition(min_val, max_val):
    samples = []
    for a in range(min_val, max_val + 1):
        for b in range(min_val, max_val + 1):
            result = a + b
            samples.append(f"{a}+{b}={result}")
    return samples

def generate_subtraction(min_val, max_val):
    samples = []
    for a in range(min_val, max_val + 1):
        for b in range(min_val, max_val + 1):
            result = a - b
            samples.append(f"{a}-{b}={result}")
    return samples

def generate_multiplication(min_val, max_val):
    samples = []
    for a in range(min_val, max_val + 1):
        for b in range(min_val, max_val + 1):
            result = a * b
            samples.append(f"{a}*{b}={result}")
    return samples

def generate_division(min_val, max_val):
    samples = []
    for a in range(min_val, max_val + 1):
        for b in range(1, max_val + 1): # avoid division by zero
            if a % b == 0:
                result = a // b
                samples.append(f"{a}/{b}={result}")
    return samples

def jumble_samples(samples):
    random.shuffle(samples)
    return samples

def main():
    samples = []
    if args.basic:

```

```

        print("\nGenerating basic single-digit math problems...")
        min_val, max_val = 0, 9
        samples += generate_addition(min_val, max_val)
        samples += generate_subtraction(min_val, max_val)
        samples += generate_multiplication(min_val, max_val)
        samples += generate_division(min_val, max_val)
    elif args.advanced:
        print("\nGenerating advanced two-digit math problems...")
        min_val, max_val = 0, 99
        samples += generate_addition(min_val, max_val)
        samples += generate_subtraction(min_val, max_val)
        samples += generate_multiplication(min_val, max_val)
        samples += generate_division(min_val, max_val)
    else:
        print("\nSpecify either --basic or --advanced")
        return

    samples = jumble_samples(samples)

    # Split samples
    split_idx = int(len(samples) * args.split)
    train_samples = samples[:split_idx]
    test_samples = samples[split_idx:]

    DATASET_DIR.mkdir(parents=True, exist_ok=True)
    TRAIN_FILE.unlink(missing_ok=True) # overwrite
    TEST_FILE.unlink(missing_ok=True) # overwrite

    print(f"Total samples: {len(samples)}")
    print(f"Train samples: {len(train_samples)}")
    print(f"Test samples: {len(test_samples)}")

    with TRAIN_FILE.open("w", encoding="utf-8") as f:
        for line in train_samples:
            f.write(line + "\n")

    with TEST_FILE.open("w", encoding="utf-8") as f:
        for line in test_samples:
            f.write(line + "\n")

    print(f"\nWrote samples to datasets/input_math_train.txt and datasets/input_math_test.txt")

if __name__ == "__main__":
    main()

```

Boolean Dataset Generation

```

#!/usr/bin/env python3

import argparse
from pathlib import Path

```

```

import random

# argparse practice
parser = argparse.ArgumentParser(description="Generate a simple boolean logic dataset.")
parser.add_argument("--basic", action="store_true", help="Generate basic boolean problems.")
parser.add_argument("--advanced", action="store_true", help="Generate advanced boolean problems.")
args = parser.parse_args()

BASE_DIR = Path(__file__).resolve().parent.parent
DATASET_DIR = BASE_DIR / "datasets"
DATASET_NAME = "input_boolean.txt"
OUTPUT_FILE = DATASET_DIR / DATASET_NAME

def eval_bool_expr(expr):
    # Convert dataset format to Python syntax for evaluation
    # ! -> not
    # & -> &
    # | -> |
    # ^ -> ^
    py_expr = expr.replace("!", "not ")
    res = eval(py_expr)
    return 1 if res else 0

def generate_basic_boolean():
    samples = []
    booleans = [1, 0]

    # NOT
    for a in booleans:
        expr = f"! {a}"
        res = eval_bool_expr(expr)
        samples.append(f"{expr}={res}")

    # AND, OR, XOR
    for op in ["&", "|", "^"]:
        for a in booleans:
            for b in booleans:
                expr = f"{a} {op} {b}"
                res = eval_bool_expr(expr)
                samples.append(f"{expr}={res}")
    return samples

def generate_complex_boolean(count=5000):
    samples = set()
    ops = ['&', '|', '^']
    booleans = [1, 0]

    max_failures = 1000
    failures = 0

```

```

while len(samples) < count:
    a = random.choice(booleans)
    b = random.choice(booleans)
    c = random.choice(booleans)

    op1 = random.choice(ops)
    op2 = random.choice(ops)

    # Patterns:
    # 1. a op1 b op2 c
    # 2. (a op1 b) op2 c
    # 3. a op1 (b op2 c)
    # 4. NOT a op1 b
    # 5. a op1 NOT b

    pattern = random.choice([1, 2, 3, 4, 5])

    if pattern == 1:
        expr = f"{a} {op1} {b} {op2} {c}"
    elif pattern == 2:
        expr = f"({a} {op1} {b}) {op2} {c}"
    elif pattern == 3:
        expr = f"{a} {op1} ({b} {op2} {c})"
    elif pattern == 4:
        expr = f"! {a} {op1} {b}"
    else:
        expr = f"{a} {op1} ! {b}"

    try:
        res = eval_bool_expr(expr)
        item = f"{expr}={res}"
        if item in samples:
            failures += 1
        else:
            samples.add(item)
            failures = 0
    except:
        pass

    if failures > max_failures:
        break

return list(samples)

def jumble_samples(samples):
    random.shuffle(samples)
    return samples

def main():
    samples = set()

```

```

if args.basic:
    print("\nGenerating basic boolean problems...")
    # Generate basic set
    basic_set = generate_basic_boolean()
    samples.update(basic_set)

elif args.advanced:
    print("\nGenerating advanced boolean problems...")
    # Include basics
    basic_set = generate_basic_boolean()
    samples.update(basic_set)

    complex_samples = generate_complex_boolean(count=10000)
    samples.update(complex_samples)
else:
    print("\nSpecify either --basic or --advanced")
    return

samples = list(samples)
samples = jumble_samples(samples)

DATASET_DIR.mkdir(parents=True, exist_ok=True)

# Split 80/20
split_idx = int(len(samples) * 0.8)
train_samples = samples[:split_idx]
test_samples = samples[split_idx:]

train_file = DATASET_DIR / "input_boolean_train.txt"
test_file = DATASET_DIR / "input_boolean_test.txt"

with train_file.open("w", encoding="utf-8") as f:
    for line in train_samples:
        f.write(line + "\n")

with test_file.open("w", encoding="utf-8") as f:
    for line in test_samples:
        f.write(line + "\n")

print(f"\nWrote {len(train_samples)} samples to {train_file}")
print(f"Wrote {len(test_samples)} samples to {test_file}\n")

if __name__ == "__main__":
    main()

```

Expression	Predicted	Expected	Result
40*95	3800	3800	✓
3+44	107	47	×
82*31	2382	2542	×
23+72	95	95	✓
28*2	56	56	✓
56+3	59	59	✓
36*47	1622	1692	×
32*84	2588	2688	×
91-18	73	73	✓
31-16	15	15	✓
52*55	2450	2860	×
18+38	56	56	✓
43-36	6	7	×
11-60	-49	-49	✓
51+41	92	92	✓
42*3	126	126	✓
50+2	52	52	✓
19+40	59	59	✓
79+0	89	79	×
17+77	94	94	✓
30*18	540	540	✓
19*7	133	133	✓
54-59	-7	-5	×
98+80	178	178	✓
97*43	4191	4171	×
50*97	4950	4850	×
86-96	-10	-10	✓
60+49	109	109	✓
37-43	-6	-6	✓
8-27	31	-19	×
41*22	802	902	×
22+80	102	102	✓
27*1	27	27	✓
44*21	994	924	×
8-43	35	-35	×
62*75	4080	4650	×
68*96	5498	6528	×
55-4	51	51	✓
97*82	7044	7954	×
16+44	60	60	✓
78-49	29	29	✓
31-52	-29	-21	×
39-81	-42	-42	✓
84*39	3896	3276	×
4+41	135	45	×
4*69	940	276	×
55*12	610	660	×
14+6	20	20	✓
40*55	2200	2200	✓
39*80	2420	3120	×

Table 6: Arithmetic evaluation: predicted vs. expected results

Expression	Pred	Exp	Result
$\neg 0 \wedge 1$	0	0	✓
$0 \wedge (1 \wedge 0)$	0	1	×
$1 \wedge 1 \wedge 1$	0	1	×
$0 \& 1 \wedge 0$	1	0	×
$(0 \mid 0) \mid 0$	0	0	✓
$1 \mid 0$	0	1	×
$1 \& (1 \wedge 0)$	0	1	×
$(1 \wedge 0) \wedge 0$	1	1	✓
$1 \& 0 \& 0$	0	0	✓
$(1 \mid 1) \& 1$	1	1	✓
$1 \mid 1$	0	1	×
$1 \wedge 0 \mid 0$	0	1	×
$0 \mid (1 \& 0)$	0	0	✓
$(0 \& 0) \wedge 0$	0	0	✓
$0 \& 1 \& 0$	0	0	✓
$0 \wedge (1 \& 0)$	0	0	✓
$1 \& 0 \mid 0$	1	0	×
$1 \wedge (0 \mid 0)$	0	1	×
$0 \wedge 1 \wedge 1$	0	0	✓
$0 \& 1 \mid 0$	1	0	×
$0 \mid 0 \wedge 0$	0	0	✓
$(1 \& 0) \wedge 1$	1	1	✓
$1 \wedge 0 \& 1$	1	1	✓
$\neg 0$	0	1	×
$\neg 1 \& 1$	1	0	×
$1 \mid (1 \wedge 0)$	1	1	✓
$0 \wedge (0 \wedge 0)$	0	0	✓
$1 \& 1 \mid 1$	0	1	×
$1 \wedge (0 \wedge 1)$	0	0	✓
$0 \& (1 \wedge 1)$	0	0	✓
$(0 \& 1) \mid 1$	1	1	✓
$(1 \& 1) \& 0$	0	0	✓
$0 \mid (0 \& 0)$	0	0	✓
$(0 \wedge 1) \& 1$	1	1	✓
$0 \wedge 0 \& 1$	1	0	×
$1 \mid 1 \wedge 0$	1	1	✓
$(1 \& 1) \mid 1$	1	1	✓
$(1 \wedge 0) \mid 0$	1	1	✓
$1 \wedge (1 \wedge 1)$	0	1	×
$0 \& 0 \wedge 0$	0	0	✓
$(0 \wedge 0) \mid 0$	0	0	✓
$0 \& (0 \wedge 0)$	0	0	✓
$1 \mid 0 \mid 1$	1	1	✓
$0 \& (1 \mid 0)$	0	0	✓
$1 \mid (1 \mid 1)$	1	1	✓
$1 \wedge 1 \& 1$	0	0	✓
$0 \mid 0 \mid 0$	1	0	×
$0 \wedge 1$	0	1	×
$(0 \wedge 0) \wedge 0$	0	0	✓

Table 7: Boolean expression evaluation: predicted vs. expected results